Computer Architecture Interview Questions

Q1. What is a CPU?

CPU (Central Processing Unit) is the main processor that executes instructions and performs calculations needed to run programs.

It contains components such as:

- Control Unit
- ALU (Arithmetic Logic Unit)
- Registers
- Cache

The CPU repeatedly **fetches, decodes, and executes instructions**.

Example:
When you open a calculator and calculate 25 + 30, the CPU executes the instructions required to perform that operation.

Q2. What is the difference between a CPU and a Core?

A CPU is the physical processor, while a **core is an individual processing unit inside the CPU**.

A CPU can have multiple cores, allowing it to execute multiple threads in parallel.

Example:

If a processor has:

CPU
├── Core 1
├── Core 2
├── Core 3
└── Core 4

it is a **4-core CPU**.

Think of it like a company:

CPU → Company
Cores → Employees doing work

More cores can allow more work to happen simultaneously, although actual performance depends on the workload and architecture.

Q3. What is RAM?

RAM (Random Access Memory) is the main volatile memory used to store data and instructions that the CPU is actively working with.

It is:

- Fast
- Volatile
- Read/write
- Temporary

When power is turned off, data in RAM is lost.

Example:

Suppose you open:

Chrome
VS Code
Spotify

Their currently needed code and data are loaded into RAM so the CPU can access them quickly.

Q4. What is the difference between RAM and ROM?

RAM is volatile read/write memory used during normal program execution.

ROM traditionally refers to non-volatile memory used for firmware or data that should persist without power.

RAM	ROM
Volatile Used actively during program execution Read/write Loses contents when powered off	Non-volatile Commonly used for firmware/storage of persistent instructions Historically read-only; modern "ROM" devices can be writable in controlled ways Retains contents without power

Example:

Your computer's:

RAM → temporarily holds running programs
Firmware/ROM-like storage → stores boot-related firmware

Q5. What is cache memory?

Cache is small, very fast memory located close to the CPU that stores frequently or recently accessed data and instructions so the CPU can access them faster than going to main memory.

The basic hierarchy is:

CPU Registers
↓
L1 Cache
↓
L2 Cache
↓
L3 Cache
↓
RAM
↓
Storage

Generally, as we move downward:

Capacity ↑
Latency ↑

while speed decreases.

Example:

Suppose the CPU repeatedly needs the same data.

Instead of repeatedly going to RAM, the processor can keep the frequently accessed data in cache.

It's like keeping frequently used books **on your desk instead of going to the library every time**.

Q6. What is the difference between L1, L2 and L3 cache?

They are different levels of CPU cache.

Cache	Typical characteristic
L1	Smallest and fastest
L2	Larger, generally slower than L1
L3	Larger, generally slower than L2 and often shared across cores

The exact sizes and organization depend on the CPU architecture.

Example:

Think of them as three storage locations:

L1 → Things on your desk
L2 → Things in your drawer
L3 → Things in your room
RAM → Things elsewhere in the house

The closer the data is to you, generally the faster it is to access.

Q7. What is a register?

A register is a very small and extremely fast storage location inside the CPU used to hold values, addresses, instructions, or other information needed during instruction execution.

Registers are generally faster to access than cache or RAM.

Example:

Suppose the CPU needs to calculate:

10 + 20

The values can be loaded into registers, the ALU performs the addition, and the result can be placed in a register.

Conceptually:

Register → 10
Register → 20
↓
ALU
↓
Register → 30

Q8. What is an instruction?

An instruction is a machine-level operation that tells the CPU what to do.

Examples include operations equivalent to:

ADD
LOAD
STORE
JUMP
COMPARE

A program is ultimately executed as a sequence of machine instructions.

Example:

For:

c = a + b;

the CPU needs multiple lower-level instructions to load values, perform the addition, and store the result.

Q9. What is a CPU cycle?

A CPU instruction cycle is the sequence of steps the processor performs to execute an instruction.

A simplified model is:

Fetch
↓
Decode
↓
Execute

Fetch

CPU gets the instruction from memory.

Decode

CPU determines what the instruction means.

Execute

CPU performs the required operation.

Example:

For an addition instruction:

Fetch → get ADD instruction
Decode → understand it's an addition
Execute → perform the addition

Q10. What is pipelining?

Pipelining is a CPU technique where the execution of multiple instructions is overlapped by dividing instruction processing into stages.

Instead of completely finishing one instruction before starting another, different instructions can occupy different stages simultaneously.

For example:

Instruction 1 → Fetch → Decode → Execute
Instruction 2 → Fetch → Decode → Execute
Instruction 3 → Fetch → Decode → Execute

Example:

Think of a car factory.

Instead of:

Build complete car 1
Build complete car 2
Build complete car 3

different workers handle different stages:

Worker 1 → Engine
Worker 2 → Body
Worker 3 → Paint

Multiple cars can be in production at the same time.

Important:
Pipelining improves **instruction throughput**; it doesn't necessarily make an individual instruction itself take fewer stages/time.

Q11. What is cache locality?

Cache locality means programs tend to access the same data again or access data located near recently accessed data.

There are two important types:

Temporal locality

If data is accessed now, it is likely to be accessed again soon.

Example:

```
for (int i = 0; i < 1000; i++) {
   total += value;
}
```

The same value is repeatedly used.

Spatial locality

If one memory location is accessed, nearby memory locations are likely to be accessed soon.

Example:

```
for (int i = 0; i < n; i++) {
   sum += arr[i];
}
```

When arr[0] is accessed, the program is likely to access:

```
arr[1]
arr[2]
arr[3]
...
```

which are stored close together in a typical contiguous array representation.

Interview tip:
This is why sequential array access often makes good use of CPU caches.

Q12. What is virtual memory?

Virtual memory is a memory-management mechanism that gives each process its own virtual address space and allows the operating system to use secondary storage as backing for memory when necessary.

It provides the illusion that a process has a large, private, contiguous memory space even though its data may actually be stored across physical RAM and, when needed, backing storage.

Example:

Suppose your computer has:

RAM = 8 GB

but running applications collectively need more memory.

The OS can move some less-active memory pages to storage, freeing RAM for active pages.

If that page is needed again, a **page fault** occurs and the OS brings it back into RAM.

This connects directly to what we discussed in Operating Systems:

Virtual Memory
↓
Paging
↓
Page Fault
↓
Storage ↔ RAM